LEXICON

NOSQL & MONGODB

Fast, safe and real

Indexes, safe Python code, a small shop app and a practice lab

LEXICON

Today

- After today you can:
  - Create indexes and check that MongoDB uses them
  - Use a unique index to stop duplicates
  - Explain operator injection and write safe filters
  - Build a small Python program on top of MongoDB
  - Know when to choose SQL and when to choose MongoDB

LEXICON

Recap

- Design by asking how the app reads the data
- Embed what belongs together, reference what is shared or grows
- Copies are fast, but must be updated everywhere (unless they should freeze)
- Schema validation = MongoDB's NOT NULL and CHECK
- Aggregation: $match, $group, $sort, $project, $unwind, $lookup

LEXICON

Quick questions

1. Write a $group stage that counts products per category.
2. When would you embed data, and when would you reference it? One example of each from the webshop.
3. What does $unwind do?

LEXICON

Indexes in MongoDB

- The same idea as in SQL: the index at the back of a book
- Without an index: MongoDB reads every document (COLLSCAN)
- With an index: it jumps straight to the right ones (IXSCAN)
- _id always has an index

```
db.products.create_index("category")
db.orders.create_index("customer_id")
```

LEXICON

Is my index used?

- totalDocsExamined: how many documents MongoDB had to look at
- nReturned: how many it gave back
- Good query: the two numbers are close

```
stats = db.big_orders.find({"customer_no": 4242}).explain()["executionStats"]
print(stats["totalDocsExamined"], stats["nReturned"])
```

LEXICON

Unique indexes

- MongoDB's version of UNIQUE
- Stops two customers with the same email

```
db.customers.create_index("email", unique=True)

db.customers.insert_one({"first_name": "Fake", "email": "anna.lindqvist@example.com"})
# DuplicateKeyError: E11000 duplicate key error
```

LEXICON

Indexes are not free

- Index the fields you search, sort or $lookup on a lot
- Don't index everything

Good	Costs
Finding documents gets much faster	Takes extra space
Sorting can get faster	Every insert and update must update the index too

LEXICON

Injection in MongoDB

- No SQL, but the same danger: user input that becomes part of the query
- Here an operator ($ne) sneaks into the filter

```
data = json.loads(request_text)       # from a web form: {"email": {"$ne": null}}
customer = db.customers.find_one({"email": data["email"]})
# becomes {"email": {"$ne": None}}: the first customer with ANY email!
```

LEXICON

Safe code

- Build the filter yourself. Never json.loads() user text into a filter
- Check the type: if not isinstance(email, str): reject it
- Convert input: str(), int(), float()
- Catch errors: DuplicateKeyError, ValueError, ServerSelectionTimeoutError

LEXICON

Personal data in MongoDB

- The same rules as in SQL: collect only what you need
- A flexible schema makes it easy to save 'just one more field'. Don't!
- Copies spread personal data: if a name is copied into 1000 orders, 'delete me' means 1000 updates
- Prefer references for personal data, and copy only what must freeze

LEXICON

SQL or MongoDB?

Choose SQL when...	Choose MongoDB when...
The data has a clear, stable structure	The structure changes often or differs between items
Many tables are combined in many ways	Data is mostly read as whole objects
Strict rules and transactions are central (money, bookings)	You need to store a lot of data fast, or spread it over many servers
You need complex reports across everything	The app already works with JSON

LEXICON

Everything in one slide

Topic	You learned
The basics	Documents, collections, _id, pymongo, find and insert
Queries	Operators, arrays, projection, sort, update and delete
Design	Embed vs reference, validation, the aggregation pipeline
Speed and safety	Indexes, safe code, a real Python app

LEXICON

Remember

- create_index() makes searches fast. explain() shows if it's used
- unique=True stops duplicates
- Never put user input straight into a filter. Check types
- Prefer references for personal data
- SQL and MongoDB are tools for different jobs

LEXICON

This afternoon: lab

Exercises 1-3 on the webshop, then the FitLife practice lab: a gym, built in MongoDB.